

Terminal

main.py | check.html x

1 <!DOCTYPE html>

2 <html>

3 <head>

4 <meta charset="utf-8">

5 <title>Check Birthday</title>

6 </head>

7 <body>

8 {% extends "template.html" %}

9 {% block content %}

10

11 <h2>{{ value[0] }}</h2>

12 <p>{{ value[1] }}</p>

13 Home

14 About

15 {% endblock %}

16 </body>

Guide x

Collapse

Backend Calculations

Working with Form Data

Start the birthday virtual environment.

conda activate birthday

The benefit of using a framework like Flask is that you can use Python for any calculations or manipulation that needs to take place (as opposed to using JavaScript). Use the link below to open `main.py`, so we can add a function to calculate if today is the user's birthday or not.

Open main.py

First update our import statements. We need to use the `datetime` package as well as `request`, which comes from `flask`.

from datetime import datetime
from flask import Flask, render_template, request

Next, we are going to create a view for the response if today is your birthday or not. Create another `app.route` decorator and name it `/check_birthday`. In addition, allow for the `POST` method so we can send data from the form to Python on the backend. Then create the function `check_birthday`.

@app.route('/check_birthday', methods=['POST'])
def check_birthday():

Use `request.form` to get information from our form. **Remember**, `'month'` and `'day'` are the names of the `<select>` tags (the drop down menus) in our form.

@app.route('/check_birthday', methods=['POST'])
def check_birthday():
 user_month = request.form['month']
 user_day = request.form['day']
 dt_obj = datetime.now()
 today_month = dt_obj.strftime('%B')
 today_day = dt_obj.strftime('%d')

Now that we know what today's date and the user's birthday, we can make a comparison. If both the day and month match, then it is the user's birthday. Either way, we need to prepare a message to send back to the website. Create two variables (`msg` and `txt`) with a message for the user. We are splitting the message into two variables so we can style them differently. As you render the `check.html` template, pass it the variable `value` and assign it a tuple with both `msg` and `text`.

@app.route('/check_birthday', methods=['POST'])
def check_birthday():
 user_month = request.form['month']
 user_day = request.form['day']
 dt_obj = datetime.now()
 today_month = dt_obj.strftime('%B')
 today_day = dt_obj.strftime('%d')

 check_month = today_month == user_month
 check_day = today_day == user_day

 if check_month and check_day:
 msg = 'Happy birthday!'
 txt = 'Today is your birthday. Spend it doing something you enjoy with those friends.'
 else:
 msg = 'It is not your birthday'
 txt = 'Sadly, today is not your birthday. Try again later.'

 return render_template('check.html', value=(msg, txt))

Message for the User

In our final template, we want to display the two messages stored in the variable `value` as well as have links for the homepage and the about page. Open the `check.html` template by clicking the link below.

Open check.html

Copy and paste the code into the IDE. **Important**, notice how a variable is surrounded by `{{ }}` in the HTML. Since `value` is a tuple, use index `0` to reference `msg` and index `1` to reference `txt`.

<!DOCTYPE html>
<html>
 <head>
 <meta charset="utf-8">
 <title>Check Birthday</title>
 </head>
 <body>
 {% extends "template.html" %}
 {% block content %}

 <h2>{{ value[0] }}</h2>
 <p>{{ value[1] }}</p>
 Home

 About
 {% endblock %}
 </body>

Open the terminal by clicking the link below. Then enter the command to start the Flask app.

Open terminal

python birthday/main.py

Then preview our app with the following link. We need to check our two use cases. Click the "Check" button without making any changes to the month or day. This should produce the message that it is not our birthday. Then enter today's date and click "Check". You should see the message about it being our birthday.

Open Flask app

In the terminal, stop the Flask app with `Ctrl + C` and then deactivate the birthday virtual environment.

conda deactivate

Reading Question

Assume the existence of the Python variable `name`. How would this variable be referenced in an HTML document using Flask?

☒

{{ name }}

☐

{ name }

☐

{{ name }}

☐

{% name %}

The correct answer is:

{{ name }}

Flask uses double curly brackets `{{ }}` in HTML to denote a Python variable.

Check it!

Next

188% (16/10)

https://codio.com/sandipan-dey/distributing-with-docker-and-heroku/tree/birthday%2Ftemplates%2Fcheck.html